

OBJECT-ORIENTED PROGRAMMING

1. INTRODUCTION TO OBJECT TECHNOLOGY

Nguyen Thi Thu Trang
trangntt@soict.hust.edu.vn



1

2

2

Outline

- ➔ 1. Programming Paradigm
- 2. Object Technology
- 3. Java
- 4. Exercises



2

33

Programming paradigm

- 1st generation
 - Machine code
- 2nd generation
 - Assembly language
 - Depending on computer architecture
- 3rd generation
 - High-level programming language
 - Procedural programming languages
 - COBOL, FORTRAN, BASIC, Pascal, C
 - Object programming languages
 - C++, Java, C#.NET, Python



3

4

1.1. Assembly code

```
;CLEAR SCREEN USING BIOS
CLR: MOV AX,0600H      ;SCROLL SCREEN
      MOV BH,30        ;COLOUR
      MOV CX,0000      ;FROM
      MOV DX,184FH     ;TO 24,79
      INT 10H          ;CALL BIOS;

;INPUTTING OF A STRING
KEY:  MOV AH,0AH        ;INPUT REQUEST
      LEA DX,BUFFER     ;POINT TO BUFFER WHERE STRING STORED
      INT 21H          ;CALL DOS
      RET              ;RETURN FROM SUBROUTINE TO MAIN PROGRAM;

; DISPLAY STRING TO SCREEN
SCR:  MOV AH,09         ;DISPLAY REQUEST
      LEA DX,STRING     ;POINT TO STRING
      INT 21H          ;CALL DOS
      RET              ;RETURN FROM THIS SUBROUTINE;
```

Assembly code

- Assembly language
 - Is a sequence programming language, is very close to machine codes of CPU.
 - Hard to remember, to write, especially for complex systems.
 - Hard to fix, to maintain.

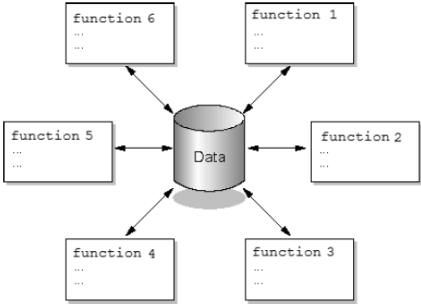


4

55

1.2. Procedural Languages

- Procedural or structural programming focuses mainly on writing software based on functions
 - Data and functions that manipulate data are kept separate
- Procedural programs cannot easily enforce a common way to access data



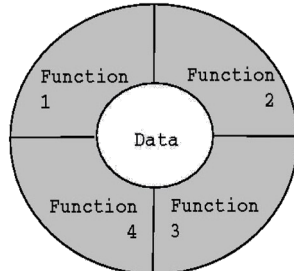
The diagram illustrates a procedural programming architecture. At the center is a cylinder labeled 'Data'. Surrounding it are six rectangular boxes, each representing a function: 'function 1', 'function 2', 'function 3', 'function 4', 'function 5', and 'function 6'. Each function box contains three dots, indicating code. Arrows point from each function box towards the central 'Data' cylinder, showing that functions interact with data but do not share a common access point.

5

66

1.3. Object-Oriented Programming

- Programming implementation technique based on objects
- Software engineering best practice
 - The use of objects allow systems to become extensible, scalable, maintainable and adaptable
- Closely represent objects of the real world, keeping the programmer in touch with the problem



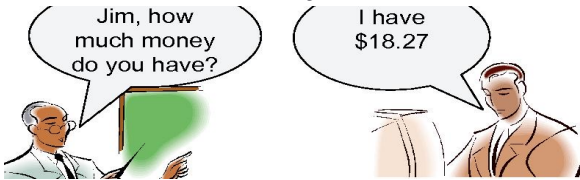
The diagram illustrates an object-oriented programming architecture. It features a central circle labeled 'Data'. Surrounding this central circle are four rectangular boxes, each representing a function: 'Function 1', 'Function 2', 'Function 3', and 'Function 4'. These boxes are arranged in a ring around the central 'Data' circle, indicating that functions are closely integrated with the data they manipulate.

6

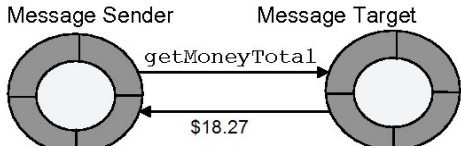
7 7

Interactions between objects

- Communication between objects in the real world



- Objects and their interactions in programming
 - Objects communicate to each other by message passing



7

8 8

Outline

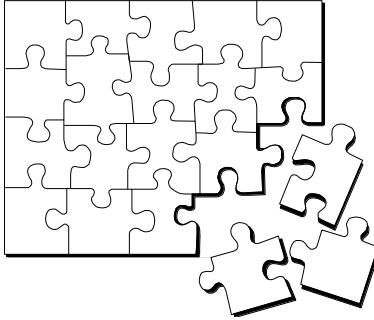
1. Programming Paradigm
- ➔ 2. Object Technology
3. Java
4. Exercises

8

99

Object Technology

- Object technology is a set of rules (e.g. abstraction, encapsulation, polymorphism), instructions to build a software, together with languages, databases and other tools to support these rules.



(Object Technology - A Manager's Guide, Taylor, 1997)



9

The Strengths of Object Technology

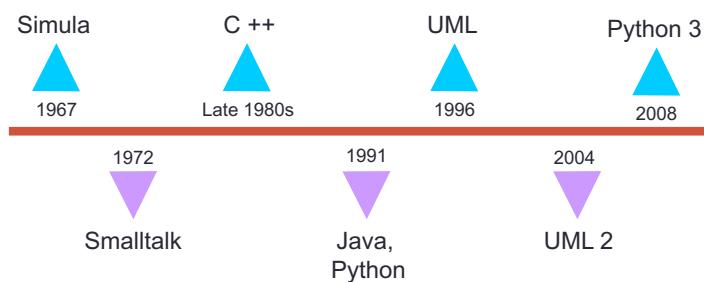
- Reflects a single paradigm
- Facilitates architectural and code reuse
- Reflects real world models more closely
- Encourages stability
- Is adaptive to change



10

The History of Object Technology

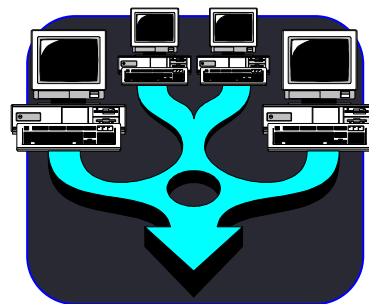
- Major object technology milestones



11

Where Is Object Technology Used?

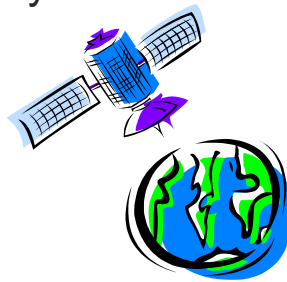
- Client/Server Systems and Web Development
 - Object technology allows companies to encapsulate business information in objects and helps to distribute processing across the Internet or a network.



12

Where Is Object Technology Used? (2)

- Real-time systems
 - Object technology enables real-time systems to be developed with higher quality and flexibility.



DEE

13

14

Object-Oriented Programming

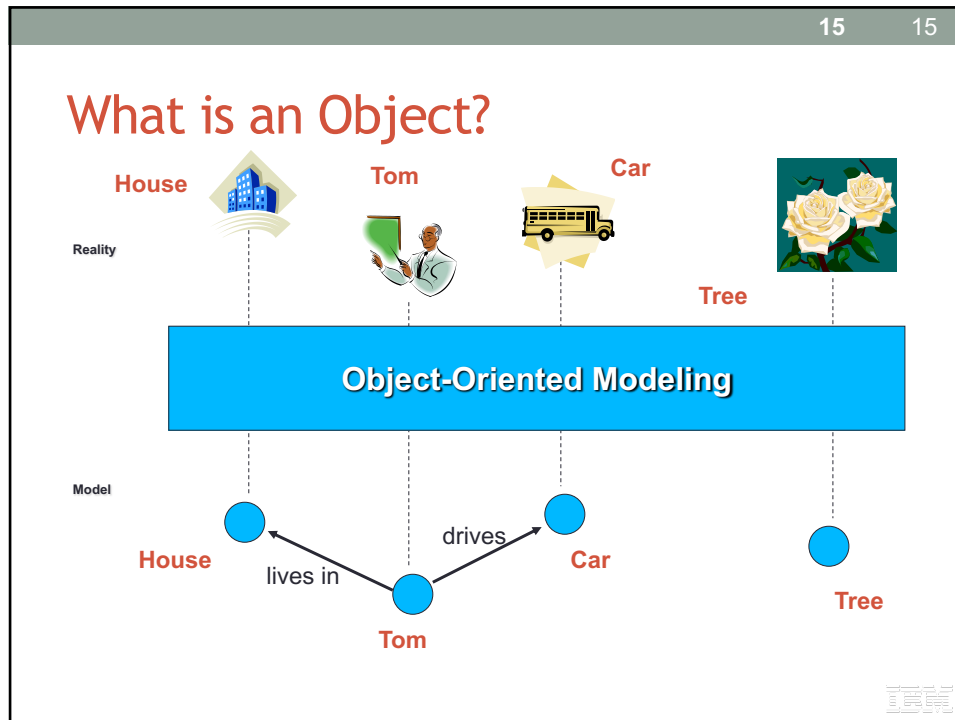
- **Object** is the key to understand the object technology
- In a OO system, all are objects



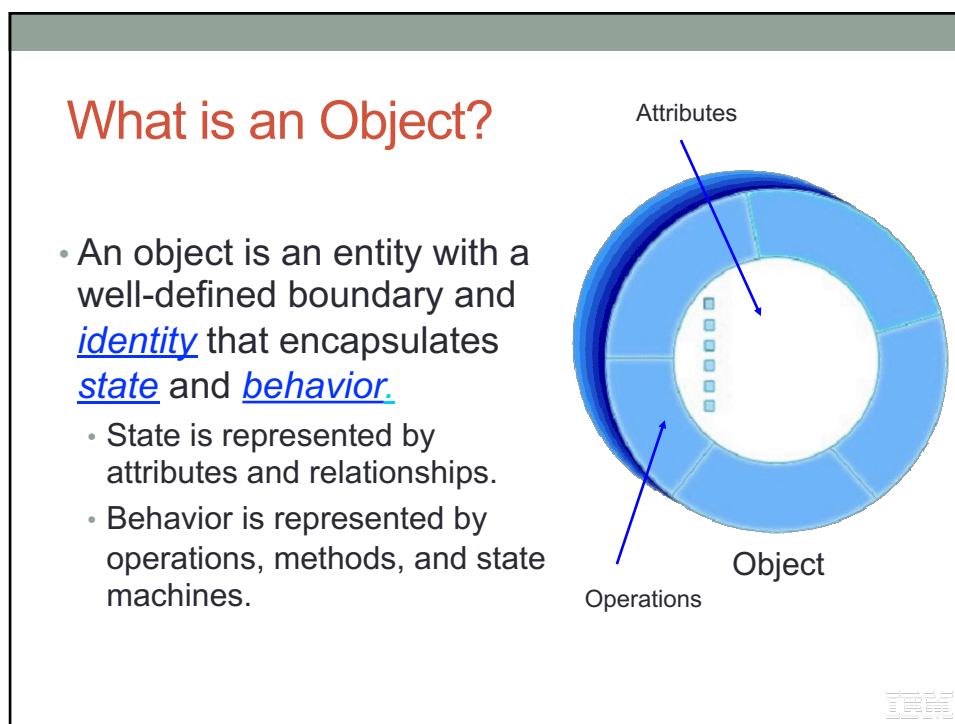
Writing a OO program means to build a model of some parts in the real world

DEE

14



15

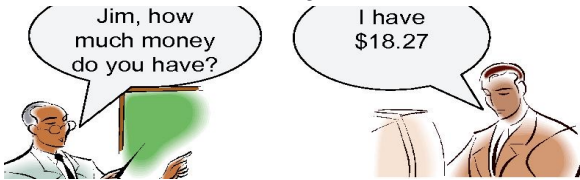


16

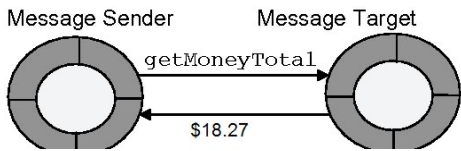
17 17

Interactions between objects

- Communication between objects in the real world



- Objects and their interactions in programming
 - Objects communicate to each other by message passing



17

18

Software objects and a real-life problem

Problem of bank account management –ATM ther – electronic payment

Attributes

Account name: Duc Binh
Balance: 2.000.000 VND

Actions

Deposit Debit

Balance inquiry



I'd like to check my balance

Service request

Balance inquiry



Account object

Attributes

name: Duc Binh
Balance: 2.000.000 VND

Actions

Deposit() Withdraw()

checkBalance()

Message passing

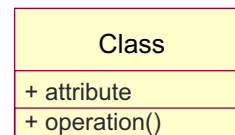
checkBalance()

User object

18

What Is a Class?

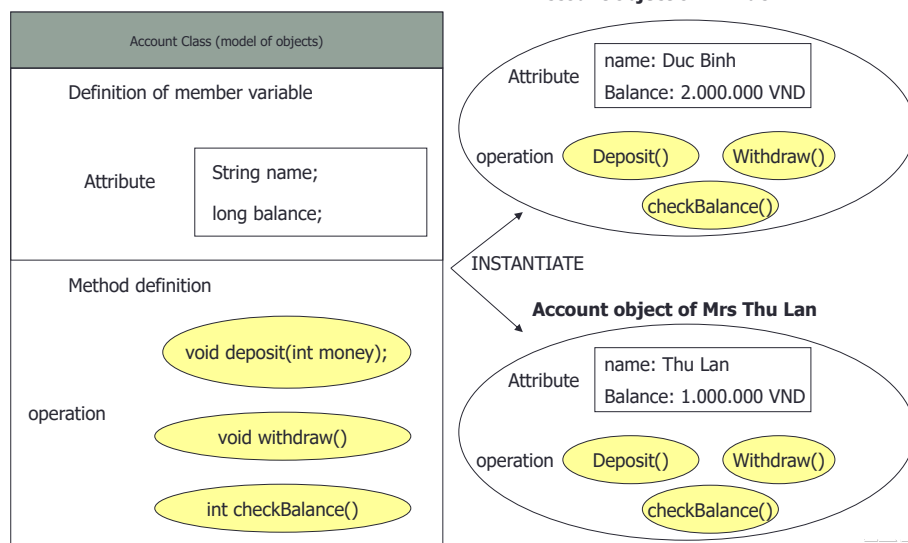
- A class is a description of a set of objects that share the same attributes, operations, relationships, and semantics.
 - An object is an instance of a class.
- A class is an abstraction in that it
 - Emphasizes relevant characteristics.
 - Suppresses other characteristics.



19

20

Class and Object



20

2121

Structural vs. OO Programming

- Structural Programming
 - Data structures + Algorithms = Program
- Object-Oriented:
 - Objects + Messages = Program

Message Sender Message Target

getMoneyTotal

\$18.27

21

21

22

Procedural vs. Object-Oriented Programming

- Procedural Programming:
 - Main components are procedures, functions
 - Data is independent with procedures
- Object-Oriented Programming
 - Main components are objects
 - Data is associated to function (method) in an object
 - Each data structure has methods executing on it

22

22

2323

Outline

1. Programming Paradigm
2. Object Technology
- ➔ 3. Java
4. Exercises



23

24

3.1 What is Java?

- Java is an object-oriented programming language developed by Sun Microsystems, and now bought by Oracle
- Java is a popular programming language
 - Initially used for building control processor applications inside the electronics consumer devices such as cell phones, microwaves
 - ...
 - Initially used in 1995



Green Team and James Gosling
(the leader)



24

2525

Java SE (Java Platform Standard Edition)

- <https://www.oracle.com/java/technologies/java-se-glance.html>
- Former name: J2SE
- Develop and deploy Java applications on desktops and servers



25

26

JRE and JDK

- Java SE Runtime Environment (JRE)
 - Executable Environment or JRE provides Java APIs, Java Virtual Machine (JVM) and other necessary components to run applets and applications written in Java.
- Java SE Development Kit (JDK)
 - Super set of JRE, and contains everything in the JRE, additional tools such as compilers and the debugger need to develop applets and applications.



26

27 27

Java EE (Java Platform Enterprise Edition)

- <https://www.oracle.com/java/technologies/java-ee-glance.html>
- Former name: J2EE. New name: Jarkata EE (from 2020)
- Extending Java SE with specifications for enterprise features such as distributed computing and web services
 - Web Applications: Servlet/JSP, JSF...
 - Enterprise Applications: EJB, JavaMail...



27

28

Java ME (Java Platform Micro Edition)

- <https://www.oracle.com/java/technologies/javame-overview.html>
- Applications running on embedded and mobile devices in the Internet of Things
 - Micro-controllers, sensors, gateways,
 - Mobile phones, personal digital assistants (PDAs),
 - TV set-top boxes, printers

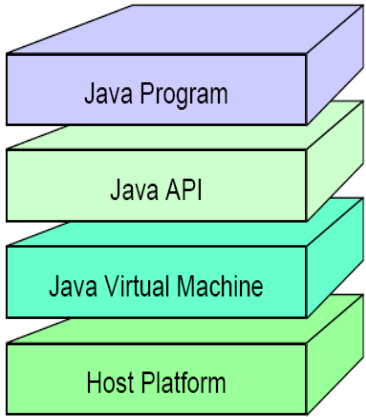


28

29 29

3.2 Java platform

- Platform is environment for development of deployment.
- Java platform can be run on all OSs
 - Other platforms depend on hardware
 - Java platform provides:
 - Java Virtual Machine (JVM).
 - Application Programming Interface (API).

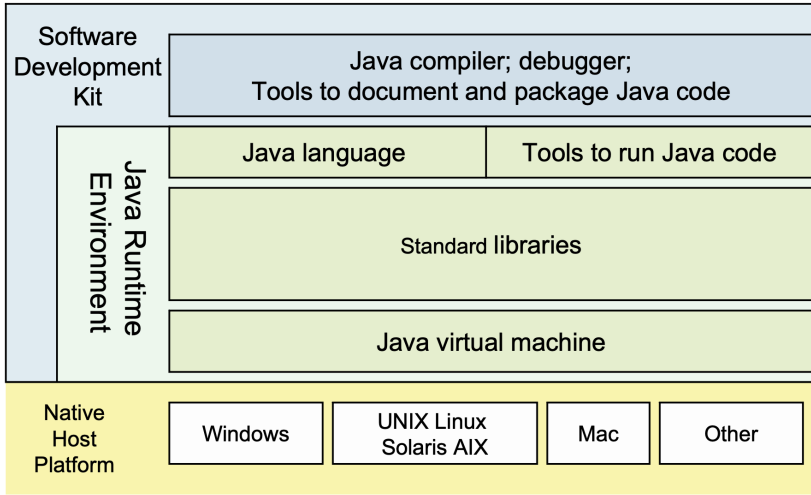




29

30

Java platform



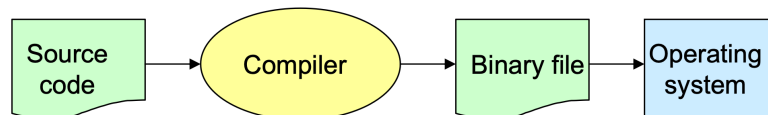


30

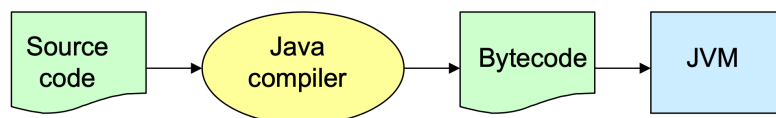
31

3.3. Compile model of Java

- **Compiled language execution model:** Source code is compiled into instructions understood by the host machine; host machine executes this binary file



- **Java execution model:** Source code is compiled into instructions understood by the JVM; bytecode is then interpreted by the JVM



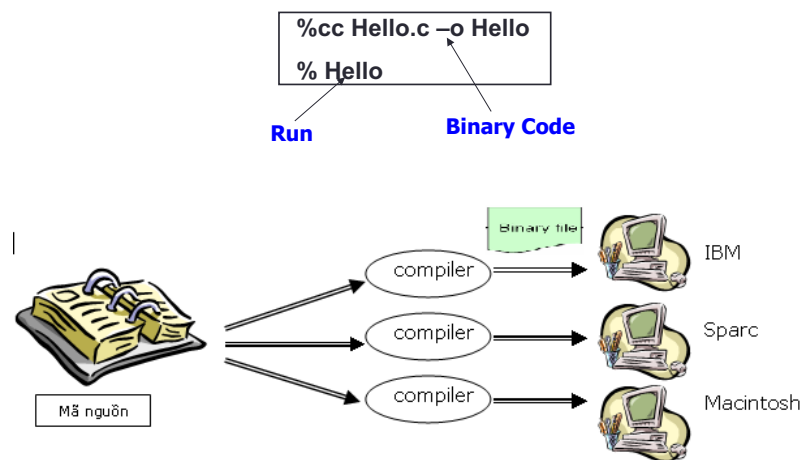
31

32

32

3.3.1. Classical compiling model

- Source code is compiled into binary code.



32

33 33

3.3. Compiling model of Java (2)

- b. Compiling model of Java:
 - Source code is compiled into bytecode and then interpreted by JVM.
 - Compiler vs. Interpreter

The diagram illustrates the Java compiling model. It shows a sequence: Source code (Hello.java) is compiled (javac) into Hello.class. Hello.class is then run (java) to produce Hello. The diagram also shows the JVM (Trình thông dịch Java) taking Bytecode (Độc lập nền - Platform independent) as input and running it on various hardware (IBM, Sparc, Macintosh).

33

34

Development procedure of Java Applications

The flowchart illustrates the development procedure of Java applications. It shows the sequence: Editor (Notepad, vi, Emacs, Editplus) -> Source code (Hello.java) -> Java Compiler (javac Hello.java) -> Hello.class -> Execution (java Hello).

34

35

Java Virtual Machine

- Write once run everywhere
- Java source is compiled into bytecode, the assembler-level code of the JVM
- JIT compilers compile bytecode into native code

```
graph LR; A[Compiled Java code] --> B[JVM JRE]; B --> C[Native operating system]; C --> D[I/O System resources];
```

- Java programmers can do things that typically require assembler-level coding on other platforms
 - Multi-threading, networking, access to resources, error handling
- Java security protects native system from Java programs

35

36 36

Java Virtual Machine

- JVM is the heart of Java language
 - Bring the feature “Write once, run everywhere”
- Provides environment to execute instructions:
 - Load file .class
 - Manage memory
 - Garbage collections
- The Interpreter “**Just In Time - JIT**”
 - JIT compilers compile bytecode into native code (machine code for each type of CPU).

```
graph LR; A[Compiled Java code] --> B[JVM JRE]; B --> C[Native operating system]; C --> D[I/O System resources];
```

36

37

37

3.4. Features of Java

- Java is designed to be:
 - A powerful programming language, full of OO features and completely OO.
 - Easy to learn, syntax is similar to C++
 - Platform independent
 - Support the development of applications in network environment
 - Ideal for Web application



37

38

38

3.4. Features of Java (2)

- Powerful
 - Class library: Hundreds of classes already written with utility functions.
 - Java uses pointer model without accessing directly to the memory; memory can not be over-written.
- Object-Oriented
 - Java supports software development by using OO
 - Software built in Java includes classes and objects



38

39

39

3.4. Features of Java (3)

- Simple
 - Keywords
 - Java has 50 keywords
 - Compared to Cobol VB that have hundreds of keywords
- Network capable
 - Java supports the development of distributed applications
 - Some applications of Java are designed in order to be accessed via Web browser.



39

40

40

3.4. Features of Java (3)

- Java has 50 key words
 - assert (New in 1.5) enum (New in 1.5)

abstract	boolean	break	byte
case	catch	char	class
const	continue	default	do
double	else	extends	final
finally	float	For	goto
If	implements	import	instanceof
int	interface	long	native
new	package	private	protected
public	return	short	static
strictfp	super	switch	synchronized
this	throw	throws	transient
try	void	volatile	while



40

4141

3.4. Features of Java (5)

- Multi-threaded
 - Allows a program to run more than one task at the same time.
- Portable
 - Programs can be written once and run on different platforms
 - Based on compiler/interpreter model (WORE – Write Once, Run Everywhere)

41

4242

3.4. Features of Java (6)

- Development Environment
 - Java Development Kit
 - Free on Sun Website: java.sun.com
 - Including: Compiler, JVM and existing classes
 - Integrated Development Environments (IDEs): Providing:
 - Complex Text Editors
 - Debugging Tools
 - Graphics Development Tools

42

4343

3.5. Applications in Java

- Application (desktop applications)
 - Do not need to run on browsers
 - Can call functions through commands or option menu (GUI)
 - main() method is the starting point of the program execution
- Applet
 - GUI application running on browser in the client side.
 - Can be viewed by appletviewer or embedded in Web browser with JVM installed.

43

4444

3.5. Applications in Java (2)

- Web applications
 - Create dynamic content on Server instead of on browsers.
 - Used in Server application
 - Servlet: manage requests from browsers and send the responses back
 - JavaServer Page (JSP): HTML pages with embedded Java code
- Mobile applications
 - Android OS

44

4545

Outline

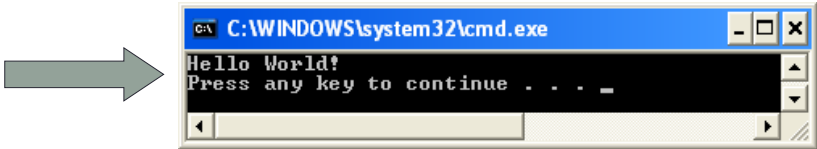
1. Object-Oriented Technology
2. Object and Class
3. Java programming languages
- ➔ 4. Examples and Exercises

45

4646

Example 1 - HelloWorld

```
// HelloWorld.java
public class HelloWorld {
    /* The main() method will be called first in any
    Java applications */
    public static void main(String args[]){
        System.out.println("Hello World!");
    }
}
```



46

4747

Example 1 (Cont.)

- Comment
 - In one line: Starts with //
 - In multiple lines: /* . . . */
- Java distinguish between lowercase and uppercase
- Keywords in Java:
 - class: Class definition
 - public: Access permission
- Class name containing main function must have the same name with the file .java.



47

4848

Installing and Running Java application

- Step 1: Install jdk, install environment variables (if using cmd)
- Step 2: Install Eclipse or Netbean IDE
- Step 3: Coding
- Step 4: Compile
 - cmd: javac HelloWorld.java
 - Eclipse/Netbean: Build automatically (Look at Console to see syntax errors if any)/F11 (Project) or F9 (File)
- Step 5: Run program
 - cmd: java HelloWorld
 - Eclipse/Netbean: Run as Java application (Alt+Shift+X+J)/F6 (Project) or Shift-F6 (File)



48

4949

Environment Variables

- PATH = %PATH%;C:\Program Files\Java\jdkx.x\bin
- JAVA_HOME=C:\Program Files\Java\jdkx.x
- CLASSPATH = C:\Program Files\Java\jdkx.x\lib;.;C:\Program Files\Java\jdkx.x\include

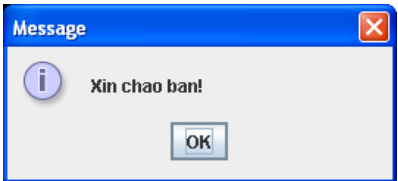


49

5050

Example 2 - GUI

```
import javax.swing.JOptionPane;  
public class FirstDialog{  
    public static void main(String[] args){  
        JOptionPane.showMessageDialog(null,  
            "Xin chao ban!");  
        System.exit(0);  
    }  
}
```



50

51 51

Example 3 - Data Input/Output

```

import javax.swing.JOptionPane;
public class HelloNameDialog{
    public static void main(String[] args){
        String result;
        result = JOptionPane.showInputDialog("Hay nhap
                                           ten ban:");

        JOptionPane.showMessageDialog(null,
                                   "Xin chao " + result + "!");

        System.exit(0);
    }
}

```

The diagram illustrates the execution of the Java program. It starts with the code, which prompts the user for their name. An arrow points to an 'Input' dialog box titled 'Input' with a question mark icon. The dialog box contains the text 'Hay nhap ten ban:' and a text field with 'Hoang' entered. Below the text field are 'OK' and 'Cancel' buttons. Another arrow points from the 'Input' dialog box to a 'Message' dialog box titled 'Message' with an information icon. The dialog box contains the text 'Xin chao Hoang!' and an 'OK' button.

51

52

Exercise

- **Write a program to calculate sum, difference, product, and quotient of 2 double numbers which are entered by users.**
- Hints:
 - To convert from String to double, you can use

```
double num1 = Double.parseDouble(strNum1)
```
 - Check the divisor of the division.

52